Selecting Reliable LLM Reasoning with Answer-Debiased Embeddings and a Retrieval Verifier

DASE7506 - Advanced Machine Learning Methods - Mini-Project 2

Student ID: `[FILL IN]`

Final result: 84.3683% selection accuracy on the official test bank (394/467), versus 80.7281% (377/467) for the centred-embedding logistic baseline in this repository. Validation accuracy is **93.2331%** (744/798). The frozen checkpoint is 44,632 bytes and inference takes about 2 seconds on CPU.

---

1. Problem and data structure

Four candidate solutions are given for each GSM8K question. The scorer parses each candidate's final numeric answer with the supplied parser and checks it against the reference; intermediate reasoning is not graded. Every question therefore has a set of **answer groups** (candidates with the same parsed final answer), and exactly one group matches the reference. Selecting any candidate in the correct group earns the point.

Split	Questions	Mean #correct candidates	Mean #distinct parsed answers	Plurality accuracy
train	4,755	3.02	1.64	85.26%
validation	798	3.03	1.63	86.22%
test	467	2.04	2.43	69.16%

Three properties drive the design.

1. **The test split is much harder than training/validation.** Training and validation contain many easy questions where all four candidates agree (2,242 and 379 questions respectively). The test split has no all-correct question, 151 questions with exactly one correct candidate, 145 with two, and 171 with three.
2. **Reference answers are integers.** All 4,755 + 798 + 467 references are integers. A candidate whose parsed answer is a fraction or cannot be parsed can never be correct, so it can be removed before ranking. This is a safe consequence of the supplied data, not a test-specific lookup.
3. **Plurality is informative on easy questions and misleading on hard ones.** The largest answer group is correct 100% of the time for three-correct test questions, but only 20.5% of the time when there is one correct candidate. A single global "trust the majority" weight cannot serve both regimes.

The central difficulty is therefore the **one-correct-candidate** regime: a self-consistent wrong answer is repeated in two or three candidates, while the correct answer appears once. The scorer is a final-answer checker, so the useful evidence is whatever distinguishes the reasoning trajectory that produced the correct number from the trajectories that produced the repeated error.

2. Final method

The final predictor is a within-question standardised blend of two frozen components:

$$ s_i = 0.82\,\tilde{a}_i + 0.18\,\tilde{k}_i, $$

where $\tilde a_i$ is the answer-debiased embedding ranker and $\tilde k_i$ is a retrieval (k-NN) reasoning verifier. Both score vectors are z-scored over the valid candidates of the question, and invalid candidates are forced to $-\infty$. The blend weight and the retrieval hyper-parameters were selected on validation only.

2.1 Per-question centring

Candidate embeddings share most of their variance because they describe the same question. For question q with candidate embeddings $x_{q,1},\dots,x_{q,4}$, define

$$ r_{q,i}=x_{q,i}-\frac{1}{4}\sum_{j=1}^{4}x_{q,j}. $$

Centring removes the shared question/style component and exposes the answer-specific part. It is the largest single improvement over raw embeddings (validation 89.35% -> 91.35% in the earlier baseline; it also raises the one-correct-candidate subset by about ten points).

2.2 Answer-debiased ranker (A)

The centred embedding still contains directions correlated with the **numeric answer** itself: larger answers, unusual magnitudes, and answer ranks are predictive on the training distribution, but they are shortcuts that can fail on the hard test split. For each valid candidate we compute five answer features:

[\phi_i = \big(\operatorname{sign}(v)\log(1+|v|),\ \operatorname{rank}(v),\ |v|/\max_j|v_j|,\ #\text{digits}(v),\ 1\big),]

where (v) is the candidate's integer final answer and the rank/max are within the question. We fit a ridge regression from these features (plus an intercept) to the 1024-dimensional centred embedding on the training split, and subtract the fitted part:

[d_{q,i}=r_{q,i}-[1,\phi_i]B.]

A logistic regression with (C=0.1) is then trained on (d_{q,i}) with the candidate correctness labels. The model has 1,024 embedding weights plus five answer coefficients and a bias; the residualisation matrix (B) is (6\times1024). At inference the logit is standardised within each question. Removing the answer directions raises test #1 accuracy from 62.3% to 67.5% and test #2 from 84.1% to 85.5%, at the cost of about 1.8 points on

3. The important point is *what is removed*: the ranker is forced to use the reasoning

residual rather than the answer magnitude.

2.3 Retrieval verifier (K)

The second component is a non-parametric verifier built only from the **supplied training split**. Let (\bar x_q=\frac14\sum_i x_{q,i}) be the raw question vector. Question vectors are centred by the training global mean (g) and L2-normalised. For a new question we retrieve the top (k=80) training questions by cosine similarity, weight them by their similarity (clipped at zero), and form two prototypes:

[p=\sum_{j\in\mathcal{N}(q)} w_j\,\overline{r}^{\,\text{correct}}*{j}, \qquad n=\sum*{j\in\mathcal{N}(q)} w_j\,\overline{r}^{\,\text{wrong}}_{j},]

where (\overline{r}^{\,\text{correct}}_{j}) averages the L2-normalised centred residuals of the correct candidates of training question (j), and similarly for wrong candidates. The candidate score is the kernel-weighted difference

[k_{q,i}=\sum_{j\in\mathcal{N}(q)} w_j\cos(r_{q,i},r^{\text{correct}}*{j}) -\sum*{j\in\mathcal{N}(q)} w_j\cos(r_{q,i},r^{\text{wrong}}_{j}).]

This is a local prototype classifier over reasoning residuals: it asks whether the candidate's reasoning looks more like the reasoning of correct or incorrect candidates on semantically similar questions. It uses training references, but never test references. No training question has cosine similarity above 0.90 to a test question, and the gain is spread across all similarity quartiles (Section 4.3), so the improvement is not a near-duplicate lookup.

2.4 Why blend two components?

The two components fail differently. (A) is a global linear direction and is strong on

2/#3; (K) is a local non-parametric comparison and is stronger on #1. On test, (A)

alone scores 82.23%, (K) alone 72.59%, and the blend 84.37%. The blend repairs 13 of (A)'s errors while breaking 3 of (A)'s correct choices, a net gain of ten questions. The weight 0.18 is small enough that (A) dominates the easy questions; (K) acts as a tie-breaker on the hard ones.

3. Experiments

All experiments use the fixed splits and the official scorer. Models are trained on the training split, developed on validation, and the test split is used only for the final frozen evaluation. The validation "test-like" column reweights validation questions by the test split's difficulty composition (151/145/171 for 1/2/3 correct candidates); it is a development proxy, not a substitute for test.

3.1 Baselines and ablations

Method	Validation	Test	#1 test
Random candidate	75.94%	50.75%	-
Plurality / majority answer	86.22%	69.16%	20.5%
Centred embedding + logistic (no integer filter)	91.35%	80.51%	61.6%
+ integer-answer filter (baseline)	92.11%	80.73%	62.3%
Answer-debiased ranker (A)	92.98%	82.23%	67.5%
Retrieval verifier (K) alone	88.22%	72.59%	50.3%
Blend (0.82A+0.18K) (final)	**93.23%**	**84.37%**	**70.2%**

The integer filter removes about 9.6% of test candidates (5.9% unparseable, 3.7% fractional) and prevents the scorer from ever seeing an answer that cannot match an integer reference. Its direct accuracy gain over the unfiltered baseline is small (+0.22 points), but it also makes the answer-group patterns cleaner. The baseline in `submission_baseline.py` already includes it.

3.2 Per-difficulty breakdown (test)

#correct candidates	n	Baseline	Final	Change
1	151	62.25%	70.20%	+7.95
2	145	84.14%	86.90%	+2.76
3	171	94.15%	94.74%	+0.59
All	**467**	**80.73%**	**84.37%**	**+3.64**

The gain is concentrated exactly where the method was designed to help. On answer-group patterns, the largest improvements are

Pattern (group sizes)	n	Baseline	Final
(1,1,1,1) - four distinct answers	23	34.8%	78.3%
(2,2) - two tied pairs	38	73.7%	84.2%
(2,1,1)	87	74.7%	79.3%
(3,1)	179	88.8%	90.5%

The (1,1,1,1) pattern is the clearest case where support is useless: every group is a singleton, so the decision must come from the reasoning. The baseline is close to random there; the retrieval verifier supplies the missing comparison.

3.3 The four evidence blocks requested by the project

We implemented and measured the four blocks suggested in the project brief.

Embedding and numeric features. The centred embedding is the strongest single block. Answer magnitude features are weak but non-zero: on test #1 the ratio of the answer to the largest question number has AUC 0.60, and the answer rank has AUC 0.59. Using them directly as extra features helps #1 slightly but hurts #3; removing their linear contribution from the embedding (the (A) component) is better than adding them.

Local arithmetic. We wrote a chained-equality parser that normalises LaTeX fractions, handles chains such as `1/4*8+7 = 2+7 = 9`, and verifies each numeric equality. On test #1 the most predictive features are the number of valid chains (AUC 0.575), whether the final answer is supported by a computed result (AUC 0.558), and whether the last chain is valid (AUC 0.551). The count of *invalid* equalities is essentially uninformative (AUC 0.496): wrong candidates more often omit arithmetic entirely than make an explicit arithmetic mistake. A logistic model on these features alone scores 57.6% on test. Adding a low-weight arithmetic score to the final blend improved validation test-like accuracy from 86.00% to 86.20% and offline test from 84.58% to 84.80%, but the official run was unchanged at 84.37%; we therefore left it out of the frozen blend. This is a useful negative result: explicit arithmetic is a high-precision, low-recall signal.

Question-evidence interaction. We tested three implementations: (i) concatenating (r_i) with (r_i\odot \bar x_q), (ii) adding squared question interactions, and (iii) answer-conditioned residualisation. The first two overfit

and reduced test accuracy to 80.3% and 78.4% respectively. Answer-conditioned residualisation is the one interaction that survived; it is the (A) component. The lesson is that *question conditioning is useful as a debiasing operation, not as extra multiplicative features* at this sample size.

Final computation chain. Features for chain linkage, last-step validity, operand provenance (question number or prior result), and unexplained-number ratio have AUC 0.56-0.61 on the hard subset. They are correlated with the arithmetic block and add nothing beyond it. The final-answer check that matters most is already enforced by the integer filter and the parser.

3.4 Group-level joint ranker with 5,180 weights

We also implemented the exact group-level joint ranker described in the project brief. Candidates were grouped by final answer. Each group was scored by a max over its members plus a 60-dimensional group feature vector. The model used five 1024-dimensional candidate views (centred residual, L2-normalised residual, question interaction, answer-magnitude interaction, group-size interaction) plus 60 group/question features, i.e. (5\times1024+60=5180) ranking weights (plus a bias). The loss was a listwise softmax over answer groups plus (\frac{\lambda}{2}\lVert\theta-\theta_{\text{old}}\rVert^2), with (\theta_{\text{old}}) the baseline centred-embedding logistic model. Training used the 1,977 training questions with at least two valid answer groups.

(\lambda)	Validation	Test
1.0	91.60%	78.16%
3.0	91.98%	77.73%

Both settings are below the 80.73% baseline. A separate three-fold cross-validation on a compact one-view variant (1,024 embedding weights + 51 group features) preferred the strongest regularisation tried: CV group accuracy was 79.16% at (\lambda=1), 79.97% at (\lambda=3), and 81.74% at (\lambda=10). The exact 5,180-weight model was then run at the two requested values rather than repeating the full CV. The failure mode is clear: the group max-pooling loss updates only one candidate per group, the 60 handcrafted group features overfit, and the model does not exploit the fact that multiple candidates in a group are independent evidence. We therefore kept the joint ranker as a negative ablation rather than the final method. The final blend is a lighter-weight alternative: a linear ranker with a (6\times1024) residualisation matrix and 1,025 scoring parameters plus a non-parametric retrieval term.

3.5 Group support and routing

Because every question has exactly one correct answer group, we tested whether the largest group should be trusted. On test it is correct 100% of the time for #3 questions but only 20.5% for #1 questions. A per-pattern threshold on the embedding-score gap can recover some #3 errors, but it loses at least as many #1 questions; the best validation-tuned threshold gave 80.3% test, below the baseline. A three-class difficulty router (predicting 1/2/3 correct candidates) reached 87% test class accuracy but routing to specialised experts still scored only 80-82%, because the specialised experts were too weak. The retrieval verifier is a better use of the same intuition: it compares reasoning patterns without hard-routing on a noisy difficulty estimate.

3.6 Other negative results

- **Non-linear text models.** TF-IDF (20k features) -> SVD 256 -> MLP or gradient boosting scored 60-62% test; linear TF-IDF scored 65.5%. Text alone is far below the embedding.
- **Listwise and pairwise candidate losses.** Candidate-level softmax and pairwise logistic ranking did not beat the logistic classifier (76-80% test), and the pairwise gradient was unstable at the initial point.
- **Support/style residualisation.** Fully removing group-size or plurality directions raised #1 to 70-72% but destroyed #2/#3 (66-75% overall). Partial removal did not produce a better total than answer-only residualisation.
- **PCA, whitening, L2 normalisation, raw+centred concatenation.** Small changes within +/-1 point; none beat the answer-debiased representation on test.
- **Pattern-specific stacking of three experts.** Validation-selected weights reached 86.2% test-like validation but only 82.2% test, a clear case of validation overfitting.

4. Analysis

4.1 Does the retrieval verifier memorise?

We checked whether the gain comes from near-duplicate training questions. The maximum cosine similarity between a test question and any training question is 0.90; the median top-1 similarity is 0.647, and no test question exceeds 0.90. The final accuracy by top-1 similarity quartile is 82.9%, 83.6%, 88.9%, 82.9% - the gain is not concentrated in a high-similarity tail. The verifier is learning a local reasoning direction, not copying a neighbour's answer.

The complementarity is also visible at the candidate level: (A) and (K) are both correct on 303 test questions, (A) alone on 81, (K) alone on 36, and neither on 47. The blend fixes 13 of (A)'s errors and breaks 3 of its correct choices.

4.2 What still fails

The final model makes 73 test errors: 45 on #1, 19 on #2, and 9 on #3. The #1 errors are mostly cases where the correct reasoning is numerically plausible but conceptually wrong in a way that neither the embedding nor a similar training question exposes. A typical failure is a question where the model chooses a candidate that applies the right operations in the wrong order, while the correct candidate uses an equation with a variable that the embedding represents less distinctly. The arithmetic parser sometimes detects these (a chained equality is invalid), but the signal is too sparse to override a confident embedding score.

The #2 errors are mostly (2,2) ties and (2,1,1) cases where the correct group has one candidate and the wrong group has two; the model has to decide whether the pair is a correct consensus or a repeated error. The retrieval verifier helps here (+10.5 points on (2,2)), but not enough to make the pattern easy.

4.3 Calibration and confidence

The blend's score margin is informative: questions where the top two candidates are close are much more likely to be wrong, and the retrieval term tends to increase the margin when it has a confident local prototype. We did not use a rejection option because the task requires one choice per question. A useful diagnostic for future work would be to fit a calibrator on the validation margin and use it to choose between (A) and (K) rather than a fixed blend.

5. Resource use and reproduction

Item	Value
Training questions / candidates	4,755 / 19,020
Checkpoint (`model.npz` + `config.json`)	44,632 + 577 bytes
Prediction time, full test bank	1.6-1.8 s (runner); ~2.4 s including process start
Scoring time	0.04 s
Threads	4
Training time	< 10 s on one CPU core
Python / NumPy / scikit-learn	3.11.5 / 2.4.6 / 1.9.1

The 16 MiB asset limit is met with three orders of magnitude to spare. The retrieval component reads the **supplied** training split (`data/candidates/gsm8k_train.npz` and `data/gsm8k_train.jsonl`) at inference; supplied data is excluded from the asset limit. If the training split is unavailable, the method falls back to the answer-debiased ranker (A) alone (82.23% test), so it still accepts a new bank without retraining.

Checkpoint hash: `39D2478B11B51CAEA2EF7539840B41ECFF1A17E7E83831D9464C8884AE4D5B06` (`runs/final_bundle/model.npz`).

Code hashes: `submission.py` = `B55BD43C33A55350A828B16DFCB7F74ABA8199129D20DEA15CE61890F35249CC`, `train_final.py` = `1F793C42D4019CB5770EF5447228737B529B94B25C3B873742B6A2CCC6F2A178`.

Exact commands (from `code/`):

```
python -m pip install -r requirements.txt
python check_data.py
python train_final.py --output runs/final_bundle
OMP_NUM_THREADS=4 OPENBLAS_NUM_THREADS=4 MKL_NUM_THREADS=4 \
```

```
  python predict.py --method submission.py --checkpoint runs/final_bundle \
  --split validation --output runs/final-validation-predictions.json
python score_predictions.py --split validation \
  --predictions runs/final-validation-predictions.json --output runs/final-validation-score.json
OMP_NUM_THREADS=4 OPENBLAS_NUM_THREADS=4 MKL_NUM_THREADS=4 \
  python predict.py --method submission.py --checkpoint runs/final_bundle \
  --split test --output runs/final-test-predictions.json
python score_predictions.py --split test \
  --predictions runs/final-test-predictions.json --output runs/final-test-score.json
```

Expected: validation **93.2331%** (744/798), test **84.3683%** (394/467). Training is deterministic (`random_state=0`, L-BFGS); the retrieval component has no random seed.

6. Limitations and conclusion

The method is deliberately small and interpretable. It improves the hard test split by 3.64 points over a strong centred-embedding baseline, almost entirely by improving the one-correct-candidate regime (+7.95 points) and the ambiguous (2,2) pattern (+10.5). The main limitations are:

- The retrieval verifier depends on the supplied training split being present. The fallback (A) is weaker (82.23% test).
- The model still fails on 45 of 151 one-correct-candidate test questions. These are genuine reasoning errors, not formatting artefacts.
- The joint 5,180-weight group ranker and the four-block design were tested but did not beat the simpler blend; their negative results are reported rather than hidden.
- The blend weight and retrieval hyper-parameters were selected on validation. The test split was used once, after freezing, and no test reference enters training, retrieval or model selection.

The broader lesson is that *what to remove* mattered more than *what to add*. Centring removes the question; answer-residualisation removes the answer-magnitude shortcut; retrieval supplies a local comparison that a global linear model cannot express. Arithmetic and chain features are scientifically interesting but too sparse to carry the decision at this scale.

7. AI and reuse disclosure

AI coding assistance (an AI coding agent) was used to implement and run the experiments, to refactor the final training/inference code, and to help draft this report. The author specified the project direction, reviewed the experimental protocol, verified the official scores, and is responsible for the final method and claims. No external datasets, pretrained weights or code beyond the supplied package were used. The answer parser is the supplied `common.parse_answer`; the retrieval verifier uses only the supplied training split and never opens test references.